

s1.03 — Thread-safe LRU cache → distributed cache

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [LRU algorithm and thread safety — IC3 baseline](#)
- [Single-flight loading, TTL, LFU, and distributed consistency — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: retain the first-hand report attribution in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: a `ConcurrentHashMap` plus a `synchronized` wrapper around a linked list, and a loader called on every miss.
 - First break: every exact-LRU hit mutates shared recency state, so a read lock is wrong and a single lock serializes all threads.
 - Second break: a slow loader returns after a newer `put` or `remove`, and its stale value silently repopulates the cache
 - The same race reappears across nodes once the cache is shared.
 - Minimum primitives
 - A map for node identity
 - A doubly linked list for `O(1)` recency
 - One lock over both
 - A per-key generation to fence asynchronous loads
 - A source version once the cache is distributed.
- 🧠 "The map gives identity, the list gives recency, one lock gives the invariant; everything after that is fencing stale work."
- Opening script
 - "I will build an in-process exact LRU first, then add thread safety, coalesced loading, TTL, and finally a cache-aside shared tier."
 - "First I confirm whether LRU must be exact, whether capacity is entries or bytes, and how much staleness callers tolerate."
 - "Then I size memory and origin load, which decides one lock versus stripes and whether single-flight is mandatory."

- “The deep dive is the map-plus-list invariant under one lock and the generation check that stops a stale load overwriting a newer write.”
- “I will close with the distributed consistency contract, failure modes, and what changes under stronger requirements.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Memory and origin load; API semantics
Architecture	17:00–30:00	Structure diagram, lock ownership, one <code>get</code> trace
Deep dive	30:00–45:00	LRU invariant, single-flight fencing, distributed staleness
Operations and tradeoffs	45:00–50:00	Failure table, stripes versus exactness, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Exact or approximate LRU?	Exact in-process; approximate acceptable when striped	Exact LRU forces one lock over map and list
Capacity unit?	Entry count first; byte weight as a follow-up	Byte weight makes one insert evict many entries
Are values mutable?	Immutable by contract	The cache cannot make caller mutations thread-safe
Who loads on a miss?	Caller-supplied loader; one origin call per key at a time	Forces single-flight, a deadline, and generation fencing
Staleness tolerance?	Seconds for reference data; zero for authorization	Decides TTL-only versus version-validated or uncached path
Deployment shape?	In-process first, then cache-aside over an authoritative database	Invalidation and source versions join the contract
Multi-tenant?	Yes, one namespace per tenant	Per-tenant byte budgets stop noisy-tenant eviction

- Functional scope
 - Start with the in-process LLD prompt.
 - `get` returns a value or miss.
 - `put` inserts or replaces a value.
 - Access and replacement move an entry to most-recent position.
 - Capacity is an entry count for the initial solution.
 - Reject null keys and values.
 - Reject nonpositive capacity.
 - Follow-ups
 - Thread safety.
 - TTL expiration.
 - Coalesced value loading.
 - LFU eviction.
 - Distributed cache-aside deployment.

Requirement	Target	Enforced by
Concurrent callers	32 application threads	One cache lock; stripes only after measured contention
Capacity	100,000 entries per instance	Eviction under the same lock as insertion
Throughput	100,000 operations/s benchmark	O(1) map/list work; no I/O inside the lock
Local operation latency	p99 < 1 ms, excluding a loader	Constant-time critical section; loader runs outside the lock
Loader deadline	200 ms	Bounded future timeout; flight removed on timeout or error
Distinct concurrent loads	At most 128	Semaphore over active flights, separate from per-key coalescing
Persistence	0 durable local cache entries required	Disposable entries; the database owns durable truth
Distributed staleness bound	TTL 30 s plus invalidation lag, disclosed to callers	TTL plus durable change-event invalidation; version check on publication

- Invariants
 - Every live key maps to exactly one list node.
 - Every non-sentinel list node appears in the map.
 - Map size equals live list length.
 - Size is at most capacity when a public operation completes.
 - A successful `get` moves the node under the same lock used by eviction.
 - A load publishes only if no newer `put`, `remove`, or `clear` happened for that key since the load was registered.
- Non-goals
 - Durable or replicated cache contents.
 - Write-back caching where the cache is the system of record.
 - Cross-tenant sharing of cached values.
 - Range queries or secondary indexes over cached keys.

Capacity and consequence — IC3 baseline

- Memory estimate
 - Assume 1 KB value and 200 B combined key/node overhead.
 - $100,000 \times 1.2 \text{ KB} \approx 120 \text{ MB}$ before table slack and JVM/allocator overhead.
 - Object layout depends on runtime configuration.
 - Measure retained heap rather than treating this estimate as a JDK constant.
 - Decision: an entry-count cap alone does not bound heap
 - Add a byte weight and a maximum entry size once values are not uniform.
- Origin impact
 - At 100,000 reads/s and 95% hit rate, origin receives 5,000 reads/s.
 - A cold cache may send 100,000 reads/s to the origin.
 - Admission control and gradual warming matter more than a perfect eviction policy during restart.
 - Decision: single-flight per key plus a 128-flight cap is mandatory, not an optimization
 - The cold-start number is what breaks the origin.
- Throughput decision
 - At 100,000 operations/s across 32 threads, one lock must hold each critical section under roughly 10 μ s to stay below saturation
 - An O(1) pointer update is well inside that budget.
 - Begin with one lock.
 - Profile lock wait time under the actual key distribution.
 - Stripe only when contention is the measured bottleneck.
 - Decision: keep exact LRU under one lock until profiling shows lock wait in the latency budget

- Stripes trade exactness for parallelism and must be a measured choice.

Architecture and flow — IC3 baseline

```
Cache facade
| one lock protects both structures
+-- hash map: key -> Node(key, value, prev, next, expiry, generation)
+-- recency list: HEAD <-> most recent <-> ... <-> least recent <-> TAIL
+-- in-flight loads: key -> Flight(future, captured generation)

Loader runs outside cache lock -> origin with deadline and concurrency bound

Distributed extension (cache-aside):
Service local cache -> shared cache shard -> authoritative database
Database transaction -> durable change event -> cache invalidation consumers
Ring / policy store (control plane) -> shard membership + tenant budgets -> service clients
```

- Cache facade owns the map/list invariant.
 - Mechanism bought: one lock makes every public operation linearizable over both structures.
 - Cost paid: every hit serializes on that lock until stripes are added.
- Loader owns origin access.
 - Mechanism bought: runs outside the lock with a deadline, so a slow origin never blocks hits.
 - Cost paid: the result can be stale on return, so publication must be fenced by generation.
- Shared cache shard owns cross-instance reuse.
 - Mechanism bought: one origin load serves many instances
 - Hit rate survives instance restarts.
 - Cost paid: a network hop on every local miss and a second place where stale values live.
- Authoritative database plus change-event stream owns truth and invalidation.
 - Mechanism bought: commit-then-invalidate with durable delivery bounds how long a stale copy survives.
 - Cost paid: invalidation is asynchronous, so a stale read is possible for a bounded window.
- Ring and policy store is the control plane.
 - Mechanism bought: shard membership and tenant budgets change without redeploying clients.
 - Cost paid: clients must cache the last-known ring so the data plane keeps serving when this store is down (static stability).
- A hash lookup returns the node directly.
 - Unlinking a known doubly linked node costs constant pointer work.
 - Searching a list for the key would cost $O(N)$.
- Expected `get` and count-bounded `put` are $O(1)$.
 - Hash-table resizing is amortized.
 - Byte-weighted insertion may evict multiple entries.
 - Lock waiting and loader execution are not covered by the data-structure complexity.

→ Acquire the cache lock.
 → Find the node by key.
 → Remove and miss if it is expired.
 → Otherwise unlink and move it to the most-recent position.
 → Return the value after releasing the lock.

API and data model — IC3 baseline

Operation	Contract	Idempotency and conflict behavior
<code>get(key)</code>	Optional value; counts as access	Idempotent for value; expiration and recency update happen under lock

Operation	Contract	Idempotency and conflict behavior
<code>put(key, value)</code>	Insert or replace; counts as access	Repeat with same value is harmless; update existing entry without increasing size
<code>remove(key)</code>	Remove current entry and invalidate an older load	Idempotent; advances generation even if the entry is absent
<code>getOrLoad(key, loader)</code>	Coalesce one in-flight load per key	Joins the existing flight; loader deadline and shared-future cancellation policy
<code>clear()</code>	Clear entries and invalidate old load generations	Idempotent; a cache epoch fences all outstanding loads
<code>stats()</code>	Approximate counters are acceptable	Do not claim a globally atomic snapshot without locking
PUT /caches/{ns}/entries/{key} (shared tier)	Value, absolute <code>expiresAt</code> , optional expected <code>source_version</code>	Version mismatch returns 409; absolute expiry keeps a retry from extending TTL
DELETE /caches/{ns}/entries/{key} (shared tier)	Invalidate	Idempotent; absent key returns 204

State	Key	Fields/invariant
Node	<code>key</code> in the local map	Value, prev, next, expiry, generation; sentinels never hold user values
Flight	<code>key</code> in the in-flight map	Shared future, captured generation, captured cache epoch; removed on success and error
Generation	<code>key</code>	Advanced by <code>put</code> and <code>remove</code> ; tombstone retained only while a flight can still reference it
Shared entry	(<code>tenant</code> , <code>namespace</code> , <code>key_hash</code>)	Bytes, <code>source_version</code> , <code>expires_at</code> ; disposable
Ring metadata	<code>ring_version</code>	Token to node mapping; clients keep the last-known version
Tenant policy	(<code>tenant</code> , <code>namespace</code>)	<code>maxBytes</code> , <code>maxEntryBytes</code> , <code>defaultTtl</code> ; versioned

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Nodes, flights, generations, and shared entries live in process or cache-node memory; only the tenant policy and the ring directory are relational.

```

CREATE TABLE cache_namespace_policies (
  tenant_id text NOT NULL, namespace text NOT NULL,
  max_bytes bigint NOT NULL CHECK (max_bytes > 0), -- per-namespace byte budget
  max_entry_bytes integer NOT NULL CHECK (max_entry_bytes > 0), -- one value cannot exhaust memory
  default_ttl_ms integer NOT NULL CHECK (default_ttl_ms > 0),
  version bigint NOT NULL CHECK (version > 0), -- If-Match validator
  updated_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, namespace),
  CHECK (max_entry_bytes <= max_bytes)
);
CREATE TABLE cache_rings (
  ring_version bigint NOT NULL, -- monotonic; clients keep the last-known
  version
  state text NOT NULL CHECK (state IN ('STAGED','ACTIVE','RETIRED')),
  created_at timestamptz NOT NULL, activated_at timestamptz,
  PRIMARY KEY (ring_version),
  CHECK ((state = 'STAGED') = (activated_at IS NULL))
);
CREATE UNIQUE INDEX cache_rings_one_active ON cache_rings (state) WHERE state = 'ACTIVE';
CREATE TABLE cache_ring_tokens (
  ring_version bigint NOT NULL, token bigint NOT NULL, -- position on the hash ring
  node_id text NOT NULL, endpoint text NOT NULL,
  PRIMARY KEY (ring_version, token),
  FOREIGN KEY (ring_version) REFERENCES cache_rings
);
CREATE INDEX cache_ring_by_node ON cache_ring_tokens (ring_version, node_id);

```

- Not captured by the DDL
 - The local map, LRU list, in-flight map, generation counters, and optional expiry heap are process memory under one lock; they restart empty.
 - Shared entries (tenant, namespace, key_hash) with bytes, source_version, expires_at live on cache nodes and are disposable; the source database owns durable truth.
 - The ring tables are cluster-wide, not tenant-scoped, because tokens map to shared nodes; tenant isolation comes from the policy row and the scoped key, not from the ring.
- Deviation from the companion
 - The companion keeps ring_version -> [(token, node_id, endpoint)] as text metadata; it is split here into cache_rings and cache_ring_tokens so a staged ring can be validated before the single ACTIVE row changes.
- State ownership
 - Node owns the key required for map removal on eviction.
 - Sentinels never contain user values and are never evicted.
 - One lock owns the map/list invariant.
 - Immutable values are the default contract.
 - Returning a mutable value does not make mutations to that object thread-safe.
- Practical alternative
 - An access-ordered LinkedHashMap can supply the same basic ordering.
 - It still requires external synchronization for concurrent structural access.
 - Access-order get changes its ordering structure.
 - See [Java 17 LinkedHashMap](#).

LRU algorithm and thread safety — IC3 baseline

```
PUT(key, value), under the cache lock:
  validate capacity, key, value, and entry size
  advance key generation for every explicit put, including absent keys
  if key exists:
    replace value
    unlink its node and add it after HEAD
    preserve the generation already advanced at operation entry
    return
  create node, insert into map, add after HEAD
  if size exceeds capacity:
    victim = TAIL.prev
    unlink victim
    remove victim.key from map
```

- Trace with capacity 2
 - `put(A,1)` gives `[A]`.
 - `put(B,2)` gives `[B,A]`.
 - `get(A)` gives `[A,B]`.
 - `put(C,3)` gives `[C,A]` and evicts B.
 - `put(A,4)` gives `[A,C]` without another eviction.
- Why a read lock is insufficient
 - A hit writes recency pointers.
 - Two readers can corrupt the list while the map remains valid.
 - `ConcurrentHashMap` protects its own structure, not the combined invariant.
- Lock discipline
 - Release in a `finally` block.
 - Avoid callbacks while holding the lock.
 - Stable, side-effect-free key hashing/equality is part of the caller contract.
 - A single internal lock does not prevent deadlocks caused by arbitrary external callbacks.
- Exact failure if the rule is broken
 - Locking the map while leaving list surgery unlocked leaves a node reachable from the map but unlinked from the list, or linked twice.
 - The next eviction then removes the wrong key or loops on a corrupted `prev/next` pair.
- Stripes
 - Each stripe owns a separate map and list.
 - Capacity and LRU order become local to each stripe.
 - A hot stripe can evict while another stripe is underutilized.

Single-flight loading, TTL, LFU, and distributed consistency — IC3 baseline

- Stampede prevention
 - Keep only one `Flight` object for an active key load.
 - Waiters wait outside the cache lock.
 - Complete futures outside the lock because completion callbacks may execute inline.
- Under lock, check cache and the in-flight map.
- Join an existing flight or register a new flight with the current key generation and cache epoch.
- Release lock and perform the bounded origin load.
- Reacquire lock and compare the flight identity, generation, and epoch.
- Install only if no intervening write, removal, or clear invalidated the load.
- Remove this exact flight from the in-flight map.
- Release lock and complete the future with the result or error.

- Stale-load rule
 - A concurrent explicit `put` wins over an older load.
 - A concurrent `remove` prevents that load from repopulating the cache.
 - If invalidated, return a current value or a documented retryable conflict.
 - Do not silently reinsert the stale origin result.
- Flight cleanup
 - Remove on success and exception.
 - A timed-out waiter must not cancel work needed by all other waiters.
 - Bound flights for different keys separately from per-key coalescing.
 - Retain generation tombstones only while needed to fence outstanding work.
- TTL
 - Check expiration against monotonic elapsed time in-process.
 - Lazy removal prevents serving expired entries.
 - Bounded background cleanup releases expired memory that is never read again.
 - TTL does not prove freshness relative to a concurrent origin update.
- LFU follow-up
 - `key -> node` stores current frequency.
 - `frequency -> recency list` resolves equal-frequency ties by LRU.
 - `minFrequency` identifies the next eviction bucket.
 - Access moves a node to the next frequency bucket.
 - If the old minimum bucket empties, advance the minimum.
 - Long-lived popularity needs aging or an admission policy to avoid retaining yesterday's hot keys forever.
- Distributed consistency boundary
 - Database remains authoritative.
 - Database commit precedes invalidation.
 - Durable change delivery reduces lost invalidations.
 - A loader that read before the commit can still race to repopulate stale data.
 - Retain an invalidation version watermark while older loads can still publish.
 - Compare that watermark and the loaded source version during conditional cache insertion.
 - Deleting the entry without retaining a version fence still allows an old loader to repopulate it.
 - TTL bounds residence time after insertion.
 - It bounds source staleness only with assumptions about source lag and load duration.
 - A short local TTL alone is not a strong-consistency protocol.
 - Distributed single-flight
 - A lease can reduce duplicate origin loads.
 - Lease expiry can permit two loaders.
 - Version checks protect publication
 - The lease is an efficiency mechanism.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Cache restart	One instance or shard	Hit-rate drop	Bounded origin fallback	Warm popular keys gradually
Origin stalls	Keys being loaded	Load deadline and flight count	Fail bounded loads; optional stale reads by policy	Clear completed flights and recover gradually
Invalidation lost	Affected key copies	Version lag or sampled mismatch	TTL expiration with disclosed staleness	Replay durable invalidation or flush affected scope

Failure	Blast radius	Detection	Degraded behavior	Recovery
Loader races with put/remove	One key	Deterministic concurrency test	Newer mutation wins	Reject stale flight publication
Scan pollutes cache	A workload cohort	Eviction rate and hit ratio	Reduced hit rate	Change admission/eviction after measuring
Mutable value changed externally	Shared readers	Invariant/assertion failures in caller	No cache-level protection	Immutable values or defensive copies
Shared cache shard unavailable	Keys hashed to that shard	Connection errors; ring health	Local cache plus circuit-broken origin fallback	Ring update; gradual rewarm of the replacement shard
Ring / policy store unavailable	No membership or budget changes	Config poll failures	Data plane serves on last-known ring and policy (static stability)	Restore store; clients reconcile to the newest <code>ring_version</code>
Single dominant hot key	One lock or one shard	Per-key access counter	Serve from a local replica of the hot value	Replicate hot keys to every instance; shorten their TTL

- Observe
 - Hit ratio per namespace and per tenant.
 - Lock wait time and critical-section duration.
 - Active flight count and loader deadline exceedances.
 - Eviction rate versus insertion rate.
 - Invalidation lag from commit to cache delete.
 - Stale-publication rejections.
- Retry discipline
 - Retry a loader only on transient origin errors, with backoff and jitter, inside the `200 ms` deadline.
 - Never retry inside the lock.
 - A `409` on a versioned shared-tier write means reload, not retry the same write.
 - Waiters on a shared flight do not each retry the origin
 - The flight retries once for all.
- Tenant isolation
 - Tenant identity is part of every key, so one tenant cannot resolve another tenant's entry.
 - Byte budgets and entry-size caps are enforced per namespace.
 - Per-tenant flight caps stop one tenant's cold start consuming the origin budget.
 - Cache metrics are labeled by tenant so a noisy tenant is attributable.
- Rollout and rollback safety
 - Serialize values with a versioned encoding
 - A rolled-back client must still decode entries written by the newer build.
 - Change ring membership by expanding first, moving ownership, then contracting.
 - Canary a new cache library on one instance
 - Compare hit ratio and lock wait before fleet rollout.
 - Deploying clients must not flush the shared tier
 - Only local caches restart.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Exact LRU with one lock	Map and ordered list	Simple linearizable operations	Recency lock contention	Interview baseline and measured moderate load
Striped LRU	Independent bounded caches	Less lock contention	No exact global LRU	Approximate eviction is acceptable

Option	What it is	Strength	Weakness	Choose when
LFU	Frequency buckets with tie ordering	Retains recurring keys	Historical frequency can dominate	Workload has stable popularity
Approximate admission/recency	Buffered access or frequency sketch	Lower contention and scan resistance	More complex policy	Production workload justifies a mature cache library
No cache	Direct origin reads	Simplest freshness semantics	Origin cost and latency	Load is small or correctness dominates

- IC4 stretch — explicit consistency choice
 - Keep ordinary reference data in a bounded-staleness cache.
 - Read critical authorization state from an authoritative or version-validated path.
 - A revocation requirement may reverse the latency-first design.
- IC4 stretch — 10× load
 - Measure skew before adding stripes.
 - A single dominant key may shift the bottleneck from eviction to value loading or serialization.
 - Separate per-tenant byte budgets to prevent a noisy tenant evicting all shared entries.
- IC4 stretch — hard byte bound
 - Reject an oversized value before allocating/publishing it.
 - Account for values being loaded and buffered outside the cache.
 - Evict-before-insert avoids a temporary resident-cache overshoot but does not bound total process memory.
- IC4 stretch — cost and blast radius
 - The shared tier's memory is the dominant cost
 - TTL and byte budgets are cost controls, not only hygiene.
 - Cell the shared tier per tenant group or region so a shard loss or a hot tenant strands one cell, not every caller.
 - Keep the local cache as a data-plane fallback so a shared-tier outage degrades hit ratio rather than availability.
- Weakest assumption
 - Immutable values and a 95% hit rate
 - A mutable value or a scan-heavy workload invalidates both the safety argument and the origin-load estimate.
 - Mitigation: enforce immutability at the API boundary and measure hit ratio per workload cohort before trusting the origin budget.
- Over-engineering warning
 - Do not build a custom lock-free eviction policy before proving the single-lock invariant.
 - Do not add a shared tier when the local hit rate already meets the origin budget.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Exact LRU, count capacity, immutable values, loader contract	Staleness tolerance per data class; what authorization must bypass	Negotiates the consistency contract instead of assuming TTL is enough
Estimates	Memory, cold-start origin load, one-lock throughput budget	Cold-start load forces single-flight and flight caps	A number turns an optimization into a requirement
Architecture	Map plus list under one lock, loader outside, cache-aside tier	Cost of each boundary; static stability against the ring store	Every component has a mechanism and a price
Deep dive	Recency under lock, generation fencing, single-flight	Source-version conditional writes; lease as efficiency, not safety	Reasons about the same race locally and across the network

Phase	IC3 must land	IC4 adds	Separating signal
Operations	Failure table, hit ratio, lock wait, invalidation lag	Tenant budgets, versioned encoding, cell blast radius	Volunteers degradation and rollout before being asked
Wrap	Recap and stripes-versus-exactness tradeoff	Names the immutability and hit-rate assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why keep the key in a list node?
 - **IC3 baseline**
 - Eviction reaches the node through the list and needs its key to remove the map entry.
 - **IC4 stretch**
 - Node identity and generation also help reject stale asynchronous work.
- Q2: Can `get` use a shared read lock?
 - **IC3 baseline**
 - Exact LRU hits modify recency.
 - **IC4 stretch**
 - Relaxed recency can move those updates off the critical path.
- Q3: A loader returns after `remove`
 - Can it insert?
 - **IC3 baseline**
 - No
 - Compare the captured generation before publishing.
 - **IC4 stretch**
 - The same issue exists across nodes and requires a source version or fencing protocol.
- Q4: The cache must enforce immediate permission revocation.
 - **IC3 baseline**
 - A TTL-only cache does not meet that premise.
 - **IC4 stretch**
 - Add authoritative version validation or remove the cache from the authorization decision.
- Q5: Does single-flight bound origin load?
 - **IC3 baseline**
 - Only for the same key
 - Also cap distinct active loads.
 - **IC4 stretch**
 - Apply tenant fairness and a total origin concurrency budget.
- Q6: Is insertion always $O(1)$?
 - **IC3 baseline**
 - Expected/amortized for count capacity with one eviction.
 - **IC4 stretch**
 - Byte-weighted insertion can evict many entries and block longer.
- Q7 (premise reversal): Why exact LRU at all instead of a `ConcurrentHashMap` with sampled eviction?
 - **IC3 baseline**
 - Exact LRU is the prompt and its invariant is provable
 - Drop exactness only when measured contention or scan pollution justifies it.
 - **IC4 stretch**
 - A frequency sketch with buffered access lowers contention and resists scans, but its eviction is probabilistic and must be validated on the real key distribution.
- Q8: Why not make the cache the system of record and write back later?

- **IC3 baseline**
 - The cache is disposable by requirement
 - Write-back turns a restart into data loss.
- **IC4 stretch**
 - Write-back needs a durable log and replication, which is a storage system, not a cache.
- Validation scenarios
 - Capacity 1 and update-existing behavior.
 - The recency trace above.
 - Map/list invariant checks after randomized operations.
 - Concurrent hit and eviction.
 - Loader exception with waiting callers.
 - `put`, `remove`, and `clear` racing with a blocked loader.
 - Expired entries retained without reads.
 - Shared-tier 409 on a stale `source_version` write.
 - Ring store unavailable while the shared tier keeps serving.
- Existing algorithm practice
 - [LRU implementation drill](#).
 - Keep the distinction between its single-thread algorithm and the synchronization contract above.

Recall card

- "A map supplies node identity; a doubly linked list supplies constant-time recency updates."
- "One lock protects both structures, including cache hits."
- "Load outside the lock and fence publication against newer mutations."
- "Distributed caching adds an origin-version and staleness contract."
- "The ring store is control plane; the cache keeps serving on the last-known ring when it is down."
- Wrap-up script
 - "The design is an exact LRU from a hash map and a doubly linked list under one lock, loads outside the lock, and a cache-aside shared tier over the authoritative database."
 - "The key invariant is that a load publishes only if its captured generation still matches, so a stale origin result never overwrites a newer write."
 - "The main tradeoff is one lock for exactness versus stripes for parallelism; I would stripe only after measuring lock wait."
 - "My weakest assumption is immutable values with a 95% hit rate; I would validate hit ratio per workload before trusting the origin budget."

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [Java 17 LinkedHashMap](#): access order and synchronization.
- [Java 17 ConcurrentHashMap](#): per-bin concurrency protects the map only, not a combined map/list invariant.
- [Cache algorithm drills](#): existing implementation practice.